Thought for 3s >

CMake 完整调用关系

入口

编译入口是 src/dsar_app/CMakeLists.txt（不是仓库根目录）。执行顺序分三个阶段：

阶段一：加载宏和配置（`include`，只定义不构建）

```
src/dsar_app/CMakeLists.txt
|
├─ include(consys/consys_utility.cmake)    ← 定义 parse_dpro_param()、import_compile_flag() 等函数
├─ include(consys/consys_ad.cmake)         ← 定义 app_product_build_marcos() 等宏
├─ include(consys/consys_bf.cmake)         ← 定义 product_build_marcos() 等宏 ★
├─ include(product/faw/faw_config.cmake)   ← FAW 公共配置: CONFIG_GE_PLAT=ON 等
|   └─ include(app_core/.../faw_config.cmake)
├─ include(dsar_plat_bf.cmake)             ← ★ Conan 平台包桥接
|   ├─ include(${CONAN_BUILD_DIRS_DSAR-PLAT-BF}/config/config.cmake)  ← 平台仓 cmake 配置
|   ├─ set(X_DOM_CAN_RT_SRC ...)           ← 设置 CAN RT SDK 源码路径
|   └─ include(pal_tool.cmake)             ← PAL 参数工具
└─ include(dsar_plat_ad.cmake)             ← AD 平台包(条件编译)
```

此时所有宏（`product_build_marcos` 等）和配置开关（`CONFIG_*`）都已经就绪，但还没有开始构建任何 target。

阶段二：构建（`add_subdirectory`）

每进入一个 `add_subdirectory` 就实际执行该目录的 CMakeLists.txt。

2.1 产品目录 — 触发 `product_build_marcos()` 宏

```
src/dsar_app/CMakeLists.txt
|
└─ add_subdirectory(product/faw/fawhq_p301/)
    |
    └─ product/faw/fawhq_p301/CMakeLists.txt  内容只有一行: product_build_marcos()
        |
        |  【宏内部执行顺序】
        |
        ├─ include(fawhq_p301_config.cmake)    ← ★ 所有 CONFIG_* 开关
        |    CONFIG_X_DOM_CAN_RT_COM = ON
        |    CONFIG_X_DOM_SOMEIP_RT_COM = ON
        |    CONFIG_APP_GE_CORE = ON
        |    CONFIG_APP_DSSAD = ON           ← DSSAD 开关
        |    CONFIG_APP_COM_CORE = ON
        |    CONFIG_APP_UDSONIP = ON
        |    ...
        |
        ├─ include(../oem_feature/oem_feature.cmake)    ← OEM 特性库
        ├─ include(autosar_adapter/.../microsar_config_com.cmake)  ← AUTOSAR COM
        ├─ include(autosar_adapter/.../microsar_config_uds.cmake)  ← AUTOSAR UDS
        ├─ include(oem_product_feature/oem_product_feature.cmake)
        |
        ├─ if(CONFIG_APP_DSSAD):
        |   include(dssad_cfg/dssad_cfg_config.cmake)
        |   └─ include(dssad_cfg/CMakeLists.txt)     ← ★ 创建 app_ge_dssad_ 静态库
        |
        ├─ add_subdirectory(proxy/)                  ← proxy 库(x_dom_can_rt/someip)
        |   ├─ add_subdirectory(x_dom_can_rt_gen_com)
        |   ├─ add_subdirectory(x_dom_someip_rt_gen_com)
        |   └─ ...
        |
        └─ add_subdirectory(app_core/)               ← ★ 进入 app_core
```

2.2 app_core — 构建 GE_CORE 和 UDSONIP

```
app_core/CMakeLists.txt
|
├─ set(GE_CORE_TARGET_NAME app_ge_core_${target_suffix})
├─ set(APP_CORE_LIB ...)
|
├─ if(CONFIG_APP_GE_CORE):
|   └─ add_subdirectory(app_ge_core/)              ← ★ GE 核心
|       |
|       └─ app_core/app_ge_core/CMakeLists.txt
|           |
|           ├─ add_library(app_ge_core_...)        ← 创建 GE_CORE 静态库
|           ├─ target_link_libraries(ge_core PRIVATE mini_dcos plat_bf_ge)  ← 基础依赖
|           |
|           ├─ if(CONFIG_APP_DSSAD):
|           |   target_link_libraries(ge_core PRIVATE app_ge_dssad_...)     ← ★ DSSAD 链入
|           |   target_include_directories(ge_core ... dssad_cfg)
|           |
|           └─ (其他条件模块: VPM, CALIB, PARAM_MANAGER...)
|
├─ if(CONFIG_APP_UDSONIP):
|   └─ add_subdirectory(app_udsonip)               ← UDS 诊断库
```

```
│
├─ if(CONFIG_APP_COM_CORE):
│    └─ add_subdirectory(app_com_core)                ← COM 核心通信库
│
└─ set(APP_CORE_LIB ${APP_CORE_LIB} PARENT_SCOPE)    ← ★ 向父作用域回传
```

2.3 回到 `product_build_marcos()` — 组装 COM SO

`product_build_marcos()` 末尾（consys_bf.cmake:209-218）：

```
# 把各模块静态库组装成 COM SO
set(product_static_lib_list                    # consys_bf.cmake:149
    ${DSAR_APP_LIB_CDD}        # plat_bf_cdd
    dcos_dcms                  # DCMS 服务框架
    ${APP_CORE_LIB}            # ← 含 app_ge_core → app_ge_dssad
    x_dom_can_rt_gen_lib_com   # CAN 信号库（SigIf_Get_* 实现）
    x_dom_someip_rt_gen_lib_com # SOMEIP 信号库
    ...)

add_library(${PRODUCT_NAME}_com SHARED ...)
target_link_libraries(${PRODUCT_NAME}_com ... ${product_static_lib_list})
                                ↑
                          最终符号在这里统一解析
```

2.4 entry — 可执行程序

```
add_subdirectory(entry/)
│
└─ entry/CMakeLists.txt
    │
    ├─ add_executable(dji_ad_app ...)
    │   target_link_libraries(dji_ad_app PRIVATE mini_dcos ...)
    │   → 运行时设置 dcos_domain = DCOS_DOMAIN
    │   → 加载 com.so（含 DSSAD）
    │
    ├─ add_executable(dji_bf_app ...)
    │   → 独立 GE 进程，也设 DCOS_DOMAIN
    │
    └─ add_executable(dji_doip_service_ipv4 ...)
       target_link_libraries(... PRIVATE mini_dcos ...)
       → 运行时设置 dcos_domain = MINIDCOS_DOMAIN
       → 加载 uds_v4.so（不含 DSSAD）
```

完整关系图

```
CMakeLists.txt (顶层)
  │
  ├─[阶段一: include, 加载宏和配置]──────────────────────────
  │   consys_utility.cmake  →  工具函数(parse_dpro_param等)
  │   consys_ad.cmake       →  app_product_build_marcos() 宏
  │   consys_bf.cmake       →  product_build_marcos() 宏 ★
  │   faw_config.cmake      →  项目级 CONFIG
  │   dsar_plat_bf.cmake    →  Conan 平台包桥接 ★
  │   dsar_plat_ad.cmake    →  AD 平台包
  │
  ├─[阶段二: add_subdirectory, 实际构建]──────────────────────
  │   │
  │   ├── product/faw/fawhq_p301/
  │   │     └── product_build_marcos()
  │   │           ├── include(fawhq_p301_config.cmake)  ← CONFIG_* 开关 ★
  │   │           ├── include(dssad_cfg/CMakeLists.txt)  ← app_ge_dssad ★
  │   │           ├── add_subdirectory(proxy)            ← CAN/SOMEIP 代理库
  │   │           └── add_subdirectory(app_core) ──────┐
  │   │                                                 │
  │   │     ◄─────────────────────────────────────────┘
  │   │     app_core/CMakeLists.txt ★
  │   │       ├── add_subdirectory(app_ge_core)  → GE_CORE_TARGET_NAME
  │   │       │     └── link: app_ge_dssad, mini_dcos, plat_bf_ge ★
  │   │       ├── add_subdirectory(app_udsonip)  → UDSONIP_TARGET_NAME
  │   │       ├── add_subdirectory(app_com_core) → COM_CORE_TARGET_NAME
  │   │       └── set(APP_CORE_LIB ... PARENT_SCOPE)
  │   │
  │   │     ◄── 回到 product_build_marcos():
  │   │           product_static_lib_list = ... APP_CORE_LIB ...
  │   │           → ${PRODUCT_NAME}_com.so  ← 最终产物 ★
  │   │
  │   ├── app_core/app_ad_core/product/faw/fawhq_p301/
  │   │     └── app_product_build_marcos()
  │   │           → ${PRODUCT_NAME}_app.so  ← AD 域 SO
  │   │
  │   ├── entry/
  │   │     → dji_ad_app, dji_bf_app, dji_doip_service_ipv4/ipv6 ★
  │   │
  │   ├── product/faw/plat_config/   → dsar_plat_config 库
  │   ├── product/common/            → dsar_plat_common_adapter 库
  │   └── tool/                      → 工具可执行程序
```

以 DSSAD 的视角走一遍

```
1. src/dsar_app/CMakeLists.txt:55
   include(consys_bf.cmake)              → product_build_marcos 宏就绪

2. src/dsar_app/CMakeLists.txt:269
   include(faw_config.cmake)             → CONFIG_GE_PLAT=ON 等

3. src/dsar_app/CMakeLists.txt:270
   include(dsar_plat_bf.cmake)           → X_DOM_CAN_RT_SRC 路径就绪
                                         → linker libs/headers 就绪

4. src/dsar_app/CMakeLists.txt:287
   add_subdirectory(product/faw/fawhq_p301/)
   → CMakeLists.txt 执行 product_build_marcos()

5. product_build_marcos() 内部:
   a. include(fawhq_p301_config.cmake)  → CONFIG_APP_DSSAD=ON
   b. include(dssad_cfg/CMakeLists.txt) → 创建 app_ge_dssad_*.a
   c. add_subdirectory(proxy/)          → 创建 x_dom_can_rt_gen_lib_com_*.a 等
   d. add_subdirectory(app_core/)
      → app_ge_core/CMakeLists.txt:
          add_library(GE_CORE_TARGET_NAME ...)
          target_link_libraries(GE_CORE PRIVATE mini_dcos plat_bf_ge)
          target_link_libraries(GE_CORE PRIVATE app_ge_dssad_*)  ← DSSAD 链入
      → APP_CORE_LIB = {app_ge_core_*, app_com_core_*}
   e. product_static_lib_list = ... APP_CORE_LIB ... x_dom_can_rt_gen_lib_com ...
   f. add_library(${PRODUCT_NAME}_com SHARED ...)
      → ${PRODUCT_NAME}_com.so ← 所有符号统一解析

6. src/dsar_app/CMakeLists.txt:316
   add_subdirectory(entry/)
   → add_executable(dji_ad_app ...)
   → 运行时加载 com.so，DSSAD 在此进程中运行(DCOS_DOMAIN)
```

一句话总结：CMake 执行是两阶段的——先 `include` 把所有宏和变量加载好，再 `add_subdirectory` 实际构建。DSSAD 在 `product_build_marcos()` 宏内部被 include 进 `app_core/app_ge_core`，最终和 CAN/SOMEIP/DCMS 库一起打包进 COM SO。关键入口在 src/dsar_app/CMakeLists.txt，核心构建逻辑在 `product_build_marcos()` 宏中。